LAB EXPERIMENTS

ITA0602-MACHINE LEARNING

BY

C DILIPKUMAR

192412133

EXP-1

1 Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

CODE:

```
# FIND-S Algorithm

import pandas as pd

# Create training dataset
data = {
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
    'Wind': ['Strong', 'Strong', 'Strong', 'Strong'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

print("Training Dataset:")
print(df)

# Initialize the most specific hypothesis
hypothesis = ['Ø'] * 6

# FIND-S algorithm
for _, row in df.iterrows():

    # Consider only positive examples
    if row['EnjoySport'] == 'Yes':
```

```
    for i, attribute in enumerate(df.columns[:-1]):

        if hypothesis[i] == 'Ø':
            hypothesis[i] = row[attribute]

        elif hypothesis[i] != row[attribute]:
            hypothesis[i] = '?'

# Display final hypothesis
print("\nMost Specific Hypothesis:")
print(hypothesis)
```

OUTPUT:

```
Training Dataset:
     Sky AirTemp Humidity    Wind Water Forecast EnjoySport
0  Sunny    Warm   Normal  Strong  Warm     Same        Yes
1  Sunny    Warm     High  Strong  Warm     Same        Yes
2  Rainy    Cold     High  Strong  Warm   Change         No
3  Sunny    Warm     High  Strong  Cool   Change        Yes

Most Specific Hypothesis:
['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the, Candidate-Elimination algorithm in python to output a description of the set of all hypotheses, consistent with the training examples

CODE:

```
# ============================================================
# EXPERIMENT: CANDIDATE-ELIMINATION ALGORITHM
# ============================================================
import pandas as pd
from itertools import product

# ------------------------------------------------------------
# STEP 1: Create Training Dataset and save it as CSV
# ------------------------------------------------------------
data = {
    'Sky':       ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'AirTemp':   ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity':  ['Normal', 'High', 'High', 'High'],
    'Wind':      ['Strong', 'Strong', 'Strong', 'Strong'],
    'Water':     ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast':  ['Same', 'Same', 'Change', 'Change'],
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
}
df = pd.DataFrame(data)
# Save dataset as CSV
df.to_csv('training_data.csv', index=False)
# Read CSV file
data = pd.read_csv('training_data.csv')

print("TRAINING DATASET")
print("=" * 70)
print(data.to_string(index=False))
# ------------------------------------------------------------
# STEP 2: Initialize Attributes
# ------------------------------------------------------------

attributes = list(data.columns[:-1])
target = data.columns[-1]
```

```
# Get possible values of every attribute
domains = {}

for attribute in attributes:
    domains[attribute] = list(data[attribute].unique())

print("\nAttributes:")
print(attributes)

# ------------------------------------------------------------
# STEP 3: Functions
# ------------------------------------------------------------

# Check whether a hypothesis covers an example
def covers(h, example):

    for i in range(len(h)):

        # Ø means no example is covered
        if h[i] == 'Ø':
            return False
        # ? accepts any value
        if h[i] != '?' and h[i] != example[i]:
            return False
    return True
# Check whether h1 is more general than or equal to h2
def more_general_or_equal(h1, h2):
    for a, b in zip(h1, h2):
        if a == '?':
            continue

        if a == b:
```

```
            continue
        return False
    return True
# Remove duplicate hypotheses
def remove_duplicates(hypotheses):

    result = []

    for h in hypotheses:
        if h not in result:
            result.append(h)

    return result
# ------------------------------------------------------------
# STEP 4: Candidate-Elimination Algorithm
# ------------------------------------------------------------

# Most specific boundary
S = [('Ø',) * len(attributes)]
# Most general boundary
G = [('?',) * len(attributes)]
# Process every training example
for index, row in data.iterrows():

    example = tuple(row[attributes])
    label = row[target]
    print("\n" + "-" * 70)
    print("Example", index + 1)
    print("Data :", example)
    print("Class:", label)
    # ======================================================
    # POSITIVE EXAMPLE
    # ======================================================
```

```
if label == 'Yes':

    # Remove G hypotheses that do not cover positive example
    G = [g for g in G if covers(g, example)
    new_S = []
    for s in S:
        if covers(s, example):
            new_S.append(s)

    else:
            # Generalize S minimally
            h = list(s)
            for i in range(len(attributes)):

                if h[i] == 'Ø':
                    h[i] = example[i]

                elif h[i] != example[i]:
                    h[i] = '?'
            h = tuple(h)
            # Keep h only if it is not more general than
            # any hypothesis in G
            valid = True

            for g in G:
                if not more_general_or_equal(g, h):
                    valid = False
                    break
            if valid:
                new_S.append(h)
    S = remove_duplicates(new_S)
```

```
    # Remove S hypotheses that are more general than
    # another S hypothesis
    final_S = []
    for s in S:
        keep = True
        for s2 in S:

            if s != s2 and more_general_or_equal(s, s2):
                keep = False
                break

        if keep:
            final_S.append(s)

    S = final_S
# ========================================================
# NEGATIVE EXAMPLE
# ========================================================
else:

    # Remove S hypotheses that cover negative example
    S = [s for s in S if not covers(s, example)]
```

OUTPUT

```
TRAINING DATASET
=============================================================================
  Sky AirTemp Humidity   Wind Water Forecast EnjoySport
Sunny    Warm   Normal Strong  Warm     Same       Yes
Sunny    Warm     High Strong  Warm     Same       Yes
Rainy    Cold     High Strong  Warm   Change        No
Sunny    Warm     High Strong  Cool   Change       Yes

Attributes:
['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']

-----------------------------------------------------------------------------
Example 1
Data : ('Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same')
Class: Yes

-----------------------------------------------------------------------------
Example 2
Data : ('Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same')
Class: Yes

-----------------------------------------------------------------------------
Example 3
Data : ('Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change')
Class: No

-----------------------------------------------------------------------------
Example 4
Data : ('Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change')
Class: Yes
```

3. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set, for building the decision tree and apply this knowledge to classify a new sample.

CODE:

```
# ============================================================

# EXPERIMENT: DECISION TREE USING ID3 ALGORITHM

# ============================================================

import pandas as pd

import math

from collections import Counter

from sklearn.tree import DecisionTreeClassifier, plot_tree

import matplotlib.pyplot as plt

# ------------------------------------------------------------

# STEP 1: Create Dataset

# ------------------------------------------------------------

data = {
```

```
    'Outlook': [
        'Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',
        'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',
        'Sunny', 'Overcast', 'Overcast', 'Rain'
    ],

    'Temperature': [
        'Hot', 'Hot', 'Hot', 'Mild', 'Cool',
        'Cool', 'Cool', 'Mild', 'Cool', 'Mild',
        'Mild', 'Mild', 'Hot', 'Mild'
    ],

    'Humidity': [
        'High', 'High', 'High', 'High', 'Normal',
        'Normal', 'Normal', 'High', 'Normal', 'Normal',
        'Normal', 'High', 'Normal', 'High],

    'Wind': [
        'Weak', 'Strong', 'Weak', 'Weak', 'Weak',
        'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
        'Strong', 'Strong', 'Weak', 'Strong'
    ],
    'PlayTennis': [
        'No', 'No', 'Yes', 'Yes', 'Yes',
        'No', 'Yes', 'No', 'Yes', 'Yes',
        'Yes', 'Yes', 'Yes', 'No'
    ]
}
df = pd.DataFrame(data)
print("TRAINING DATASET")
print("=" * 70)
print(df.to_string(index=False))
# ------------------------------------------------------------
```

```
# STEP 2: Entropy Function
# ------------------------------------------------------------
def entropy(target):
    counts = Counter(target)
    total = len(target)

    result = 0

    for count in counts.values():
        probability = count / total
        result -= probability * math.log2(probability)
    return result

# ------------------------------------------------------------
# STEP 3: Information Gain Function
# ------------------------------------------------------------

def information_gain(data, attribute, target):

    total_entropy = entropy(data[target])

    weighted_entropy = 0

    for value in data[attribute].unique():

        subset = data[data[attribute] == value]

        weighted_entropy += (
            len(subset) / len(data)
        ) * entropy(subset[target])

    return total_entropy - weighted_entropy
# ------------------------------------------------------------
```

```
# STEP 4: Calculate Information Gain
# ------------------------------------------------------------

print("\n" + "=" * 70)
print("INFORMATION GAIN")
print("=" * 70)

attributes = list(df.columns[:-1])

for attribute in attributes:

    gain = information_gain(df, attribute, 'PlayTennis')

    print(f"{attribute:15} : {gain:.4f}")



# ------------------------------------------------------------
# STEP 5: ID3 Decision Tree
# ------------------------------------------------------------

def id3(data, attributes, target):

    # If all examples belong to one class
    if len(data[target].unique()) == 1:
        return data[target].iloc[0]
    # If no attributes remain
    if len(attributes) == 0:
        return data[target].mode()[0]

    # Find attribute with maximum information gain
    gains = {}

    for attribute in attributes:
```

```
        gains[attribute] = information_gain(
            data, attribute, target
        )

    best_attribute = max(gains, key=gains.get)

    tree = {best_attribute: {}}

    remaining_attributes = [
        attribute for attribute in attributes
        if attribute != best_attribute
    ]

    # Create branches
    for value in data[best_attribute].unique():

        subset = data[data[best_attribute] == value]

        if subset.empty:
            tree[best_attribute][value] = data[target].mode()[0]

        else:
            tree[best_attribute][value] = id3(
                subset,
                remaining_attributes,
                target
            )
    return tree
# Build tree
decision_tree = id3(
    df,
    attributes,
    'PlayTennis'
```

```
)

print("\n" + "=" * 70)
print("ID3 DECISION TREE")
print("=" * 70)

print(decision_tree)
# ------------------------------------------------------------
# STEP 6: Classification Function
# ------------------------------------------------------------

def classify(tree, sample):

    # If tree directly contains a class
    if not isinstance(tree, dict):
        return tree
  attribute = next(iter(tree))

    value = sample[attribute]

    subtree = tree[attribute][value]

    return classify(subtree, sample)
# ------------------------------------------------------------
# STEP 7: Classify a New Sample
# ------------------------------------------------------------

new_sample = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}
```

```
prediction = classify(
    decision_tree,
    new_sample
)
print("\n" + "=" * 70)
print("CLASSIFICATION OF NEW SAMPLE")
print("=" * 70)
print("New Sample:")
print(new_sample)
print("\nPredicted Class:", prediction)
# ------------------------------------------------------------
# STEP 8: Visualize Decision Tree
# ------------------------------------------------------------
# Convert categorical data to numerical values
df_encoded = df.copy()
for column in df_encoded.columns:
    df_encoded[column] = df_encoded[column].astype('category').cat.codes
X = df_encoded.drop('PlayTennis', axis=1)
y = df_encoded['PlayTennis']

# Train sklearn ID3-equivalent decision tree
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)
model.fit(X, y)
plt.figure(figsize=(14, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=['No', 'Yes'],
    filled=True,
    rounded=True
```

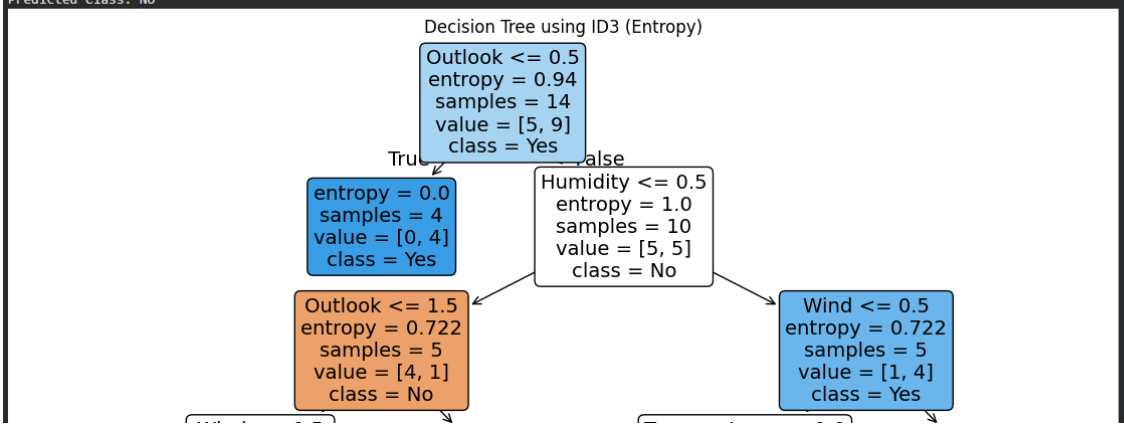
)

plt.title("Decision Tree using ID3 (Entropy)")

plt.show()

OUTPUT:

```
TRAINING DATASET
====================================================================
 Outlook Temperature Humidity   Wind PlayTennis
   Sunny         Hot     High   Weak         No
   Sunny         Hot     High Strong         No
Overcast         Hot     High   Weak        Yes
    Rain        Mild     High   Weak        Yes
    Rain        Cool   Normal   Weak        Yes
    Rain        Cool   Normal Strong         No
Overcast        Cool   Normal Strong        Yes
   Sunny        Mild     High   Weak         No
   Sunny        Cool   Normal   Weak        Yes
    Rain        Mild   Normal   Weak        Yes
   Sunny        Mild   Normal Strong        Yes
Overcast        Mild     High Strong        Yes
Overcast         Hot   Normal   Weak        Yes
    Rain        Mild     High Strong         No

====================================================================
INFORMATION GAIN
====================================================================
Outlook         : 0.2467
Temperature     : 0.0292
Humidity        : 0.1518
Wind            : 0.0481

====================================================================
ID3 DECISION TREE
====================================================================
{'Outlook': {'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}, 'Overcast': 'Yes', 'Rain': {'Wind': {'Weak': 'Yes', 'Strong': 'No'}}}}

====================================================================
CLASSIFICATION OF NEW SAMPLE
====================================================================
New Sample:
{'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}

Predicted Class: No
```



4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the, same using appropriate data sets.

CODE:

```
# ============================================================
# EXPERIMENT: ARTIFICIAL NEURAL NETWORK USING BACKPROPAGATION
# ============================================================
import numpy as np
# ------------------------------------------------------------
# STEP 1: Training Dataset
# XOR Dataset
# ------------------------------------------------------------
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)
Y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)
# ------------------------------------------------------------
# STEP 2: Initialize Network Parameters
# ------------------------------------------------------------

np.random.seed(42)

input_neurons = 2
hidden_neurons = 4
output_neurons = 1

# Weights
```

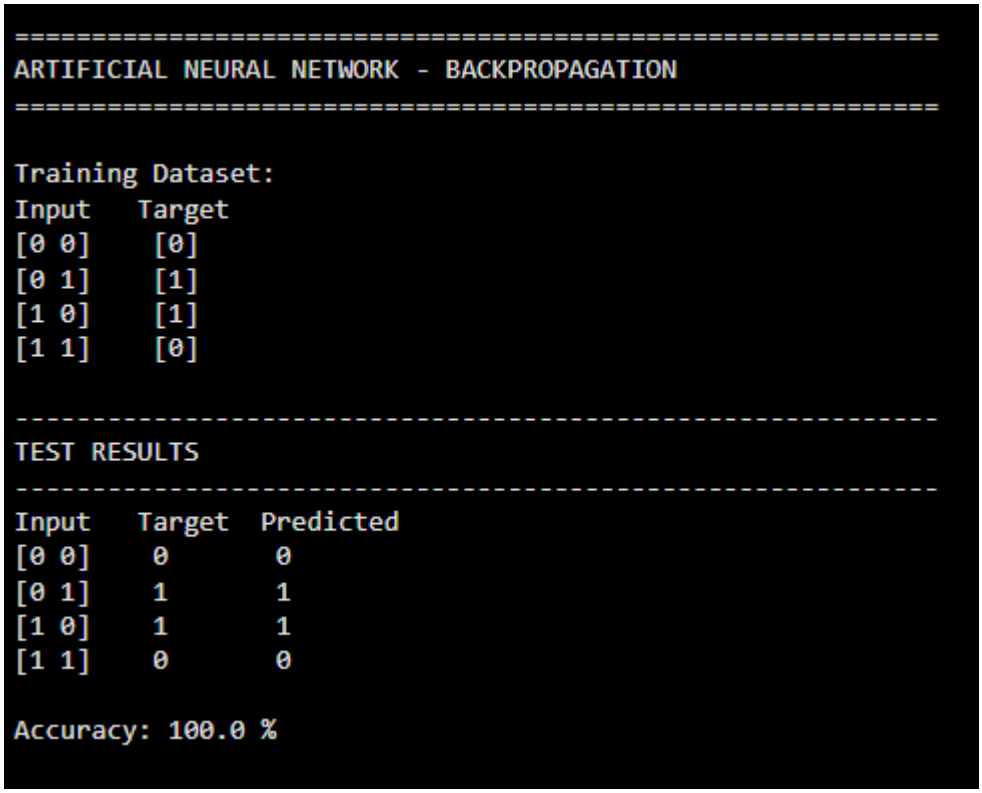
```
W1 = np.random.randn(input_neurons, hidden_neurons)
W2 = np.random.randn(hidden_neurons, output_neurons)

# Biases
b1 = np.zeros((1, hidden_neurons))
b2 = np.zeros((1, output_neurons))



# ------------------------------------------------------------
# STEP 3: Activation Function
# ------------------------------------------------------------

def sigmoid(x):
    return 1 / (1 + np.exp(-x))



def sigmoid_derivative(x):
    return x * (1 - x)



# ------------------------------------------------------------
# STEP 4: Backpropagation Training
# ------------------------------------------------------------

learning_rate = 0.5
epochs = 10000

for epoch in range(epochs):

    # -------------------------
    # Forward Propagation
    # -------------------------
```

```
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    output_input = np.dot(hidden_output, W2) + b2
    output = sigmoid(output_input)
    # --------------------------
    # Calculate Error
    # --------------------------

    error = Y - output
    # --------------------------
    # Backpropagation
    # --------------------------
    output_delta = error * sigmoid_derivative(output)

    hidden_error = np.dot(output_delta, W2.T)

    hidden_delta = hidden_error * sigmoid_derivative(hidden_output)
    # --------------------------
    # Update Weights and Biases
    # --------------------------

    W2 += learning_rate * np.dot(hidden_output.T, output_delta)
    b2 += learning_rate * np.sum(output_delta, axis=0, keepdims=True)

    W1 += learning_rate * np.dot(X.T, hidden_delta)
    b1 += learning_rate * np.sum(hidden_delta, axis=0, keepdims=True)



# ------------------------------------------------------------
# STEP 5: Test the Trained Neural Network
# ------------------------------------------------------------
```

```
hidden_input = np.dot(X, W1) + b1
hidden_output = sigmoid(hidden_input)
output_input = np.dot(hidden_output, W2) + b2
predicted_output = sigmoid(output_input)
# Convert probabilities into 0 or 1
predicted_class = (predicted_output >= 0.5).astype(int)
# ------------------------------------------------------------
# STEP 6: Display Results
# ------------------------------------------------------------

print("=" * 60)
print("ARTIFICIAL NEURAL NETWORK - BACKPROPAGATION")
print("=" * 60)

print("\nTraining Dataset:")
print("Input\tTarget")

for i in range(len(X)):
    print(X[i].astype(int), "\t", Y[i].astype(int))
print("\n" + "-" * 60)
print("TEST RESULTS")
print("-" * 60)

print("Input\tTarget\tPredicted")

for i in range(len(X)):
    print(
        X[i].astype(int),
        "\t",
        int(Y[i][0]),
        "\t",
        int(predicted_class[i][0])
    )
```

```
# ------------------------------------------------------------

# STEP 7: Calculate Accuracy

# ------------------------------------------------------------

accuracy = np.mean(predicted_class == Y) * 100


print("\nAccuracy:", accuracy, "%")
```

OUTPUT:



5. Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

CODE:

```
#
# EXPERIMENT: IMPLEMENTATION OF K-NEAREST NEIGHBOURS (K-NN)
#
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
# ------------------------------------------------------------
# STEP 1: Load Dataset
# ------------------------------------------------------------
iris = load_iris()
X = iris.data
Y = iris.target
print("=" * 60)
print("IRIS DATASET")
print("=" * 60)

print("\nFeatures:")
print(iris.feature_names)

print("\nClasses:")
print(iris.target_names)

print("\nDataset Shape:", X.shape)



# ------------------------------------------------------------
```

```
# STEP 2: Split Dataset into Training and Testing Data
# ------------------------------------------------------------

X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42)
print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))
# ------------------------------------------------------------
# STEP 3: Feature Scaling
# ------------------------------------------------------------

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# ------------------------------------------------------------
# STEP 4: Create K-NN Model
# ------------------------------------------------------------
K = 5
model = KNeighborsClassifier(n_neighbors=K)

# Train the model
model.fit(X_train, Y_train)
# ------------------------------------------------------------
# STEP 5: Predict Test Data
# ------------------------------------------------------------

Y_pred = model.predict(X_test)
# ------------------------------------------------------------
# STEP 6: Calculate Accuracy
```

```
# ------------------------------------------------------------
accuracy = accuracy_score(Y_test, Y_pred)

print("\n" + "=" * 60)
print("K-NN RESULTS")
print("=" * 60)

print("\nK value:", K)

print("Accuracy:", accuracy * 100, "%")

# ------------------------------------------------------------
# STEP 7: Display Predictions
# ------------------------------------------------------------

print("\nActual vs Predicted:")
print("-" * 40)

for actual, predicted in zip(Y_test, Y_pred):

    print(
        "Actual:",
        iris.target_names[actual],
        " | Predicted:",
        iris.target_names[predicted]
    )

# ------------------------------------------------------------
# STEP 8: Confusion Matrix
# ------------------------------------------------------------

cm = confusion_matrix(Y_test, Y_pred)
```

```
print("\nConfusion Matrix:")
print(cm)

# ------------------------------------------------------------
# STEP 9: Predict a New Sample
# ------------------------------------------------------------

# New flower:
# [Sepal Length, Sepal Width, Petal Length, Petal Width]

new_sample = np.array([
    [5.1, 3.5, 1.4, 0.2]
])
# Scale the new sample
new_sample_scaled = scaler.transform(new_sample)

# Predict
prediction = model.predict(new_sample_scaled)
print("\n" + "=" * 60)
print("NEW SAMPLE CLASSIFICATION")
print("=" * 60)
print("\nNew Sample:", new_sample[0])
print(
    "Predicted Class:",
    iris.target_names[prediction[0]]
)
```

OUTPUT:

```
============================================================
IRIS DATASET
============================================================

Features:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Classes:
['setosa' 'versicolor' 'virginica']

Dataset Shape: (150, 4)

Training Samples: 120
Testing Samples : 30

============================================================
K-NN RESULTS
============================================================

K value: 5
Accuracy: 100.0 %

Actual vs Predicted:
--------------------------------------
Actual: versicolor  | Predicted: versicolor
Actual: setosa  | Predicted: setosa
Actual: virginica  | Predicted: virginica
Actual: versicolor  | Predicted: versicolor
Actual: versicolor  | Predicted: versicolor
Actual: setosa  | Predicted: setosa
Actual: versicolor  | Predicted: versicolor
Actual: virginica  | Predicted: virginica
Actual: versicolor  | Predicted: versicolor
Actual: versicolor  | Predicted: versicolor
Actual: virginica  | Predicted: virginica
Actual: setosa  | Predicted: setosa
Actual: setosa  | Predicted: setosa
Actual: setosa  | Predicted: setosa
Actual: setosa  | Predicted: setosa
Actual: versicolor  | Predicted: versicolor
Actual: virginica  | Predicted: virginica
Actual: versicolor  | Predicted: versicolor
Actual: versicolor  | Predicted: versicolor
Actual: virginica  | Predicted: virginica
Actual: setosa  | Predicted: setosa
Actual: virginica  | Predicted: virginica
Actual: setosa  | Predicted: setosa
Actual: virginica  | Predicted: virginica
Actual: virginica  | Predicted: virginica
Actual: virginica  | Predicted: virginica
Actual: virginica  | Predicted: virginica
Actual: virginica  | Predicted: virginica
Actual: setosa  | Predicted: setosa
Actual: setosa  | Predicted: setosa

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

============================================================
NEW SAMPLE CLASSIFICATION
============================================================

New Sample: [5.1 3.5 1.4 0.2]
Predicted Class: setosa
```

6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

code:

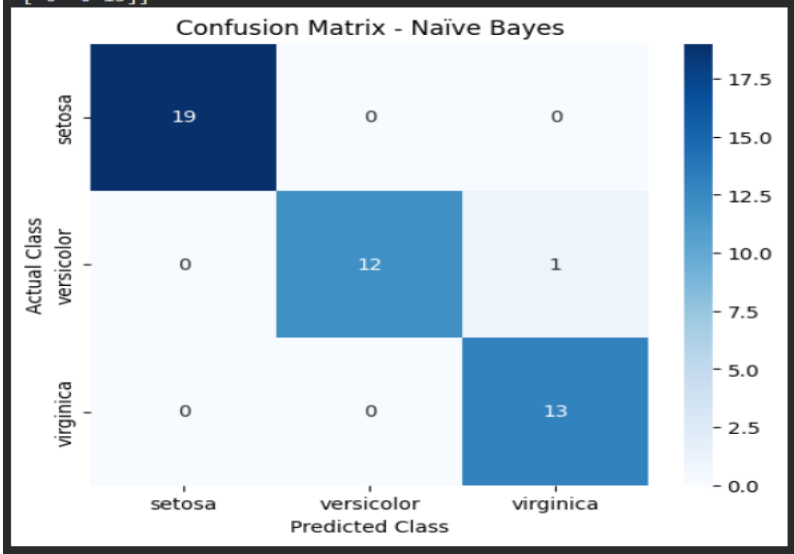
```
# EXP 7: Naïve Bayes Algorithm
# Implement Naïve Bayes and display Confusion Matrix and Accuracy
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

# Create Naïve Bayes model
model = GaussianNB()

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)
```

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Actual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix - Naïve Bayes")
plt.show()
```

```
# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

output:

```
Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 2 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 0.9777777777777777
Accuracy Percentage: 97.77777777777777 %

Confusion Matrix:
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```




```
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      0.92      0.96        13
   virginica       0.93      1.00      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45
```

7. Write a program to implement Logistic Regression (LR) algorithm in python

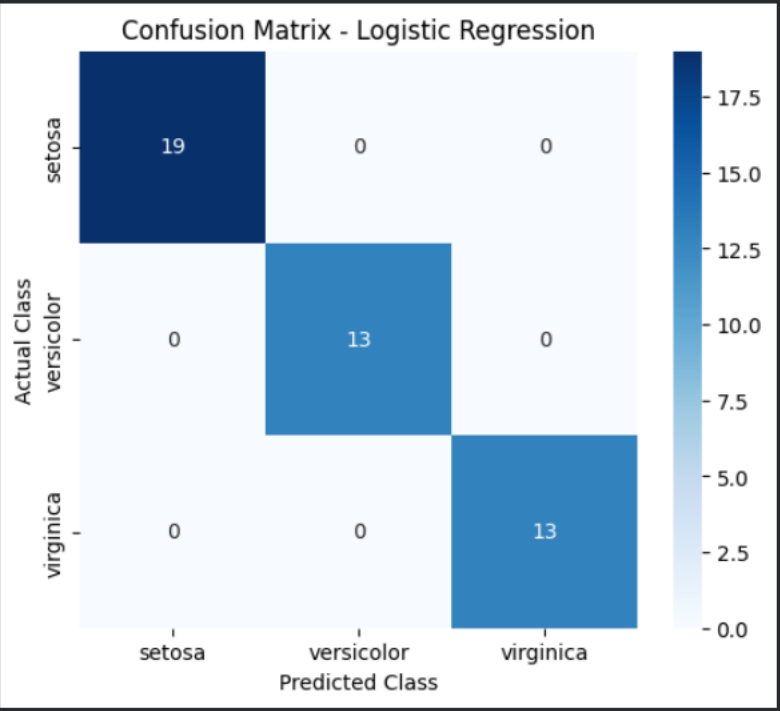
code:

```
# EXP 7: Logistic Regression (LR)
# Implement Logistic Regression algorithm in Python

# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)
# Create Logistic Regression model
model = LogisticRegression(max_iter=200)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Actual Values:")
```

```
print(y_test)
print("\nPredicted Values:")
print(y_pred)
print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
# Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()

# Classification report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

output:

```
Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 1.0
Accuracy Percentage: 100.0 %

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```




```
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

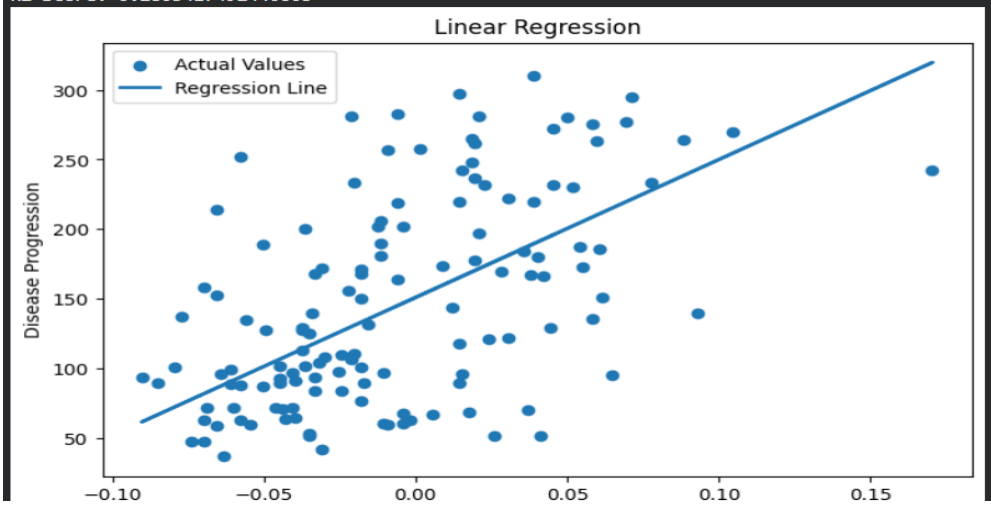
8. Write a program to implement Linear Regression (LR) algorithm in python code:

```
# EXP 8: Linear Regression (LR)
# Implement Linear Regression algorithm in Python
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error, r2_score
# Load dataset
data = load_diabetes()
X = data.data[:, [2]]   # BMI feature
y = data.target
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)
# Create Linear Regression model
model = LinearRegression()
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Display coefficients
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)
# Display actual and predicted values
print("\nActual Values:")
print(y_test)
print("\nPredicted Values:")
```

```
print("\nMean Squared Error:", mse)
print("R2 Score:", r2)
# Plot regression line
plt.figure(figsize=(8, 5))
plt.scatter(X_test, y_test, label="Actual Values")
plt.plot(X_test, y_pred, linewidth=2, label="Regression Line")
plt.xlabel("BMI")
plt.ylabel("Disease Progression")
plt.title("Linear Regression")
plt.legend()
plt.show()
```

output:

```
 118.27468285  93.77208666  90.57609585 165.14921469  95.9027472
 156.62657253 221.61171895 238.65700325 178.99850818 208.82775572
 189.65181087 108.68671042 102.29472881 173.67185684 194.97846222
 165.14921469 209.89308599 133.18930661  77.79213262 129.99331581
 242.91832433 114.01336177 165.14921469 144.90793957 190.71714114
 228.00370056 120.40534338 118.27468285 120.40534338  93.77208666
  82.0534537  121.47067365 128.92798554 118.27468285 106.55604989
 116.14402231 115.07869204 101.22939854  67.13882993 152.36525146
 208.82775572  82.0534537  168.34520549 110.81737096 133.18930661
 215.21973733 105.49071962 212.02374652 133.18930661  96.96807747
 181.12916872 191.78247141 204.56643464 106.55604989  86.31477477
 169.41053576 139.58128823 126.797325   117.20935258 138.51595796
 133.18930661 181.12916872 129.99331581 140.6466185   90.57609585
 111.88270123 210.95841625 189.65181087 170.47586603 131.05864608
 147.03860011 139.58128823  87.38010504  91.64142612  86.31477477
 125.7319473   88.44543531  82.0534537  186.45582007 163.01855415
 133.18930661 200.30511356 116.14402231  86.31477477 111.88270123
 171.5411963  219.48105841 188.5864806  134.25463688  83.11878397
 170.47586603  93.77208666 254.63695729  72.46548128 171.5411963
 110.81737096 116.14402231 107.62138016 170.47586603 176.86784764
 115.07869204 192.84780168 101.22939854 205.63176491 165.14921469
 139.58128823 140.6466185  141.71194877 147.03860011 166.21454495
 174.73718711 114.01336177 119.34001312 169.41053576 135.31996715
 144.90793957 159.82256334 114.01336177 141.71194877  74.59614182
 149.16926065 106.55604989 196.04379249]

Mean Squared Error: 3884.936720961032
R2 Score: 0.2803417492440603
```



9. Compare Linear and Polynomial Regression using Python code:

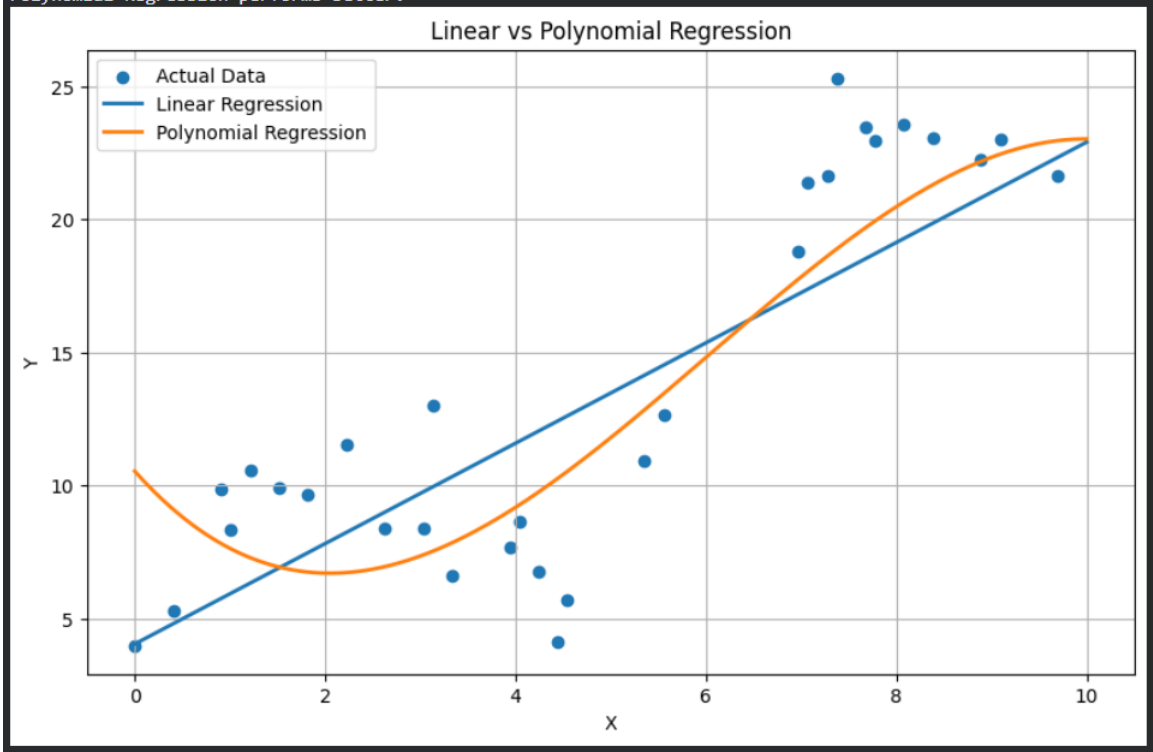
```
# EXP 9: Comparison of Linear and Polynomial Regression

# Import required libraries

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import PolynomialFeatures

from sklearn.metrics import mean_squared_error, r2_score


# Create sample dataset

np.random.seed(42)


X = np.linspace(0, 10, 100).reshape(-1, 1)

y = 2 * X.ravel() + 3 + 5 * np.sin(X.ravel()) + np.random.normal(0, 2, 100)


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.30, random_state=42

)


# ---------- Linear Regression ----------

linear_model = LinearRegression()

linear_model.fit(X_train, y_train)


y_pred_linear = linear_model.predict(X_test)


linear_mse = mean_squared_error(y_test, y_pred_linear)

linear_r2 = r2_score(y_test, y_pred_linear)


# ---------- Polynomial Regression ----------
```

```
poly = PolynomialFeatures(degree=3)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)

y_pred_poly = poly_model.predict(X_test_poly)

poly_mse = mean_squared_error(y_test, y_pred_poly)
poly_r2 = r2_score(y_test, y_pred_poly)

# Display results
print("LINEAR REGRESSION")
print("Mean Squared Error:", linear_mse)
print("R2 Score:", linear_r2)

print("\nPOLYNOMIAL REGRESSION")
print("Mean Squared Error:", poly_mse)
print("R2 Score:", poly_r2)

# Compare models
print("\nCOMPARISON")
if poly_r2 > linear_r2:
    print("Polynomial Regression performs better.")
else:
    print("Linear Regression performs better.")
# Plot results
plt.plot(X_curve, linear_curve, linewidth=2, label="Linear Regression")
plt.plot(X_curve, poly_curve, linewidth=2, label="Polynomial Regression")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear vs Polynomial Regression")
```

```
plt.legend()

plt.grid(True)

plt.show()
```

output:

```
LINEAR REGRESSION
Mean Squared Error: 14.579826823141603
R2 Score: 0.7096608139589748

POLYNOMIAL REGRESSION
Mean Squared Error: 10.805778148094479
R2 Score: 0.7848163170856082

COMPARISON
Polynomial Regression performs better.
```



10. Write a Python Program to Implement Expectation & Maximization Algorithm

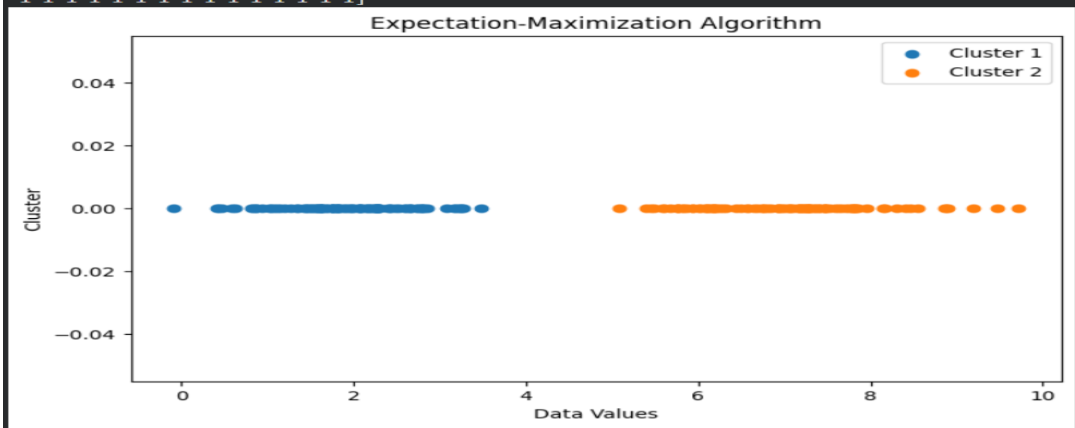
code:

```
# EXP 10: Expectation-Maximization (EM) Algorithm
# Implement EM algorithm using Gaussian Mixture Model
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
# Generate sample data
np.random.seed(42)
X1 = np.random.normal(2, 0.8, 100)
X2 = np.random.normal(7, 1.0, 100)
X = np.concatenate([X1, X2]).reshape(-1, 1)
# Create Gaussian Mixture Model
em_model = GaussianMixture(
    n_components=2,
    random_state=42
)

# Train the model using Expectation-Maximization
em_model.fit(X)
# Predict cluster labels
labels = em_model.predict(X)
# Display parameters
print("Means of the clusters:")
print(em_model.means_)
print("\nCovariances of the clusters:")
print(em_model.covariances_)
print("\nMixing weights:")
print(em_model.weights_)

# Display cluster assignments
print("\nCluster Labels:")
```

```
print(labels)

# Plot the clusters

plt.figure(figsize=(8, 5))

plt.scatter(

    X[labels == 0],

    np.zeros_like(X[labels == 0]),

    label="Cluster 1"

)

plt.scatter(

    X[labels == 1],

    np.zeros_like(X[labels == 1]),

    label="Cluster 2"

)

plt.xlabel("Data Values")

plt.ylabel("Cluster")

plt.title("Expectation-Maximization Algorithm")

plt.legend()

plt.show()
```

output:

```
Means of the clusters:
[[1.91671617]
 [7.02169685]]

Covariances of the clusters:
[[[0.52241622]]

 [[0.90253713]]]

Mixing weights:
[0.49992024 0.50007976]

Cluster Labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
```



11. Write a program for the task of Credit Score Classification

code:

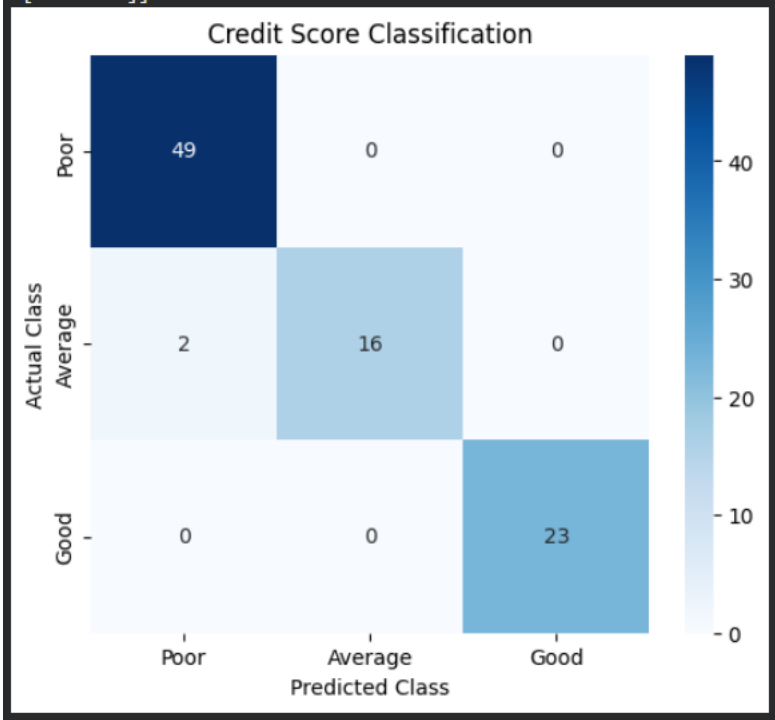
```
# EXP 11: Credit Score Classification
# Python program to classify credit scores using Random Forest
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Create sample credit score dataset
np.random.seed(42)
data = {
    "Age": np.random.randint(18, 65, 300),
    "Annual_Income": np.random.randint(20000, 120000, 300),
    "Loan_Amount": np.random.randint(5000, 100000, 300),
    "Credit_Utilization": np.random.uniform(0.1, 0.9, 300),
    "Payment_History": np.random.randint(1, 11, 300)
}
df = pd.DataFrame(data)
# Create credit score classes
def classify_credit(row):
    score = (
        row["Payment_History"] * 50
        + row["Annual_Income"] / 10000
        - row["Loan_Amount"] / 20000
        - row["Credit_Utilization"] * 20
    )

    if score >= 400:
```

```
        return "Good"
    elif score >= 300:
        return "Average"
    else:
        return "Poor"
df["Credit_Score"] = df.apply(classify_credit, axis=1)
# Features and target
X = df.drop("Credit_Score", axis=1)
y = df["Credit_Score"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
# Create Random Forest classifier
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Actual Values:")
print(y_test.values)
print("\nPredicted Values:")
print(y_pred)
print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

```
# Confusion Matrix
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=["Poor", "Average", "Good"]
)
print("\nConfusion Matrix:")
print(cm)
# Display confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Poor", "Average", "Good"],
    yticklabels=["Poor", "Average", "Good"]
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Credit Score Classification")
plt.show()

# Classification report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        labels=["Poor", "Average", "Good"]
    )
)
```

output:

```
 'Poor' 'Poor' 'Poor' 'Poor' 'Good' 'Poor' 'Poor' 'Poor' 'Poor' 'Poor'
 'Average' 'Poor' 'Poor' 'Average' 'Average' 'Poor' 'Average' 'Average'
 'Poor' 'Poor' 'Poor' 'Good' 'Average' 'Good' 'Poor' 'Average' 'Good'
 'Poor' 'Poor' 'Poor' 'Good' 'Poor' 'Good' 'Poor' 'Good' 'Average' 'Good'
 'Good' 'Poor' 'Good' 'Poor' 'Poor' 'Poor' 'Poor' 'Good' 'Poor' 'Good'
 'Poor' 'Good' 'Good' 'Poor' 'Average' 'Poor' 'Poor' 'Poor' 'Poor' 'Poor'
 'Good' 'Average' 'Poor' 'Good' 'Good' 'Poor' 'Poor' 'Poor' 'Average'
 'Good' 'Poor' 'Good' 'Average' 'Good' 'Poor' 'Poor' 'Average' 'Poor'
 'Good' 'Poor' 'Poor' 'Poor' 'Good' 'Good']

Accuracy: 0.9777777777777777
Accuracy Percentage: 97.77777777777777 %

Confusion Matrix:
[[49  0  0]
 [ 2 16  0]
 [ 0  0 23]]
```




```
Classification Report:
              precision    recall  f1-score   support

        Poor       0.96      1.00      0.98        49
     Average       1.00      0.89      0.94        18
        Good       1.00      1.00      1.00        23

    accuracy                           0.98        90
   macro avg       0.99      0.96      0.97        90
```

Exp 12

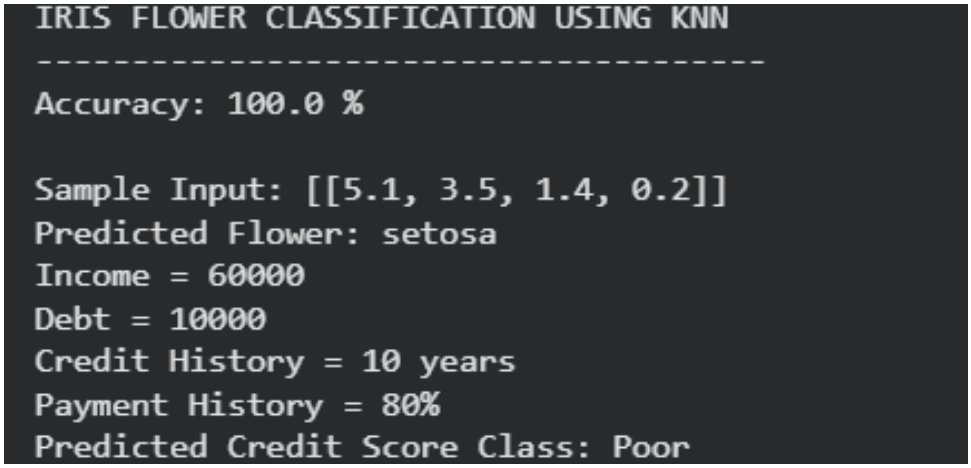
code

EXPERIMENT 12

```
# EXPERIMENT 12
# IRIS FLOWER CLASSIFICATION USING KNN

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=5)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print("IRIS FLOWER CLASSIFICATION USING KNN")
print("-------------------------------------")
print("Accuracy:", round(accuracy * 100, 2), "%")

# Test sample
sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(sample)

print("\nInput:", sample)
print("Predicted Flower:", iris.target_names[prediction[0]])
```

output:



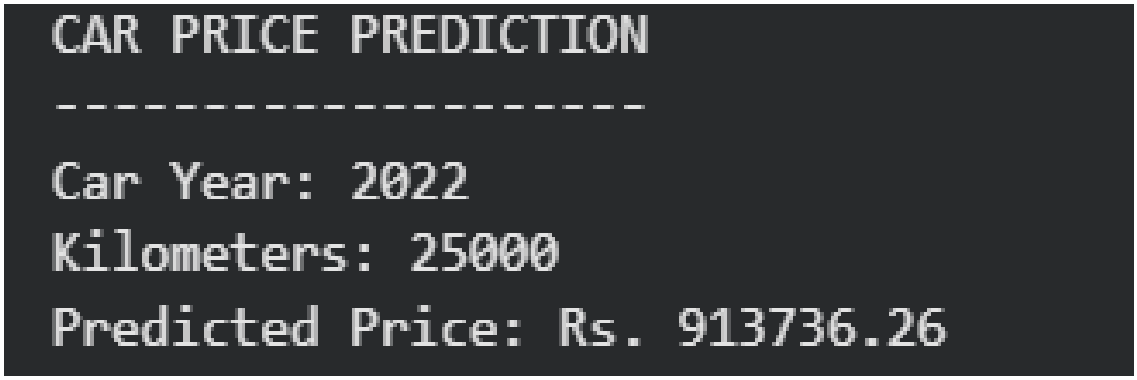
EXP 13

CODE:

```
# EXPERIMENT 13
# CAR PRICE PREDICTION

import numpy as np
from sklearn.linear_model import LinearRegression

# Create sample dataset
year = np.array([
    2015, 2016, 2017, 2018, 2019,
    2020, 2021, 2022, 2023, 2024
])

kilometers = np.array([
    90000, 80000, 70000, 60000, 50000,
    40000, 30000, 20000, 15000, 10000
])

price = np.array([
    400000, 450000, 500000, 550000, 600000,
    700000, 800000, 900000, 1000000, 1100000
])

# Input features
X = np.column_stack((year, kilometers))

# Target
y = price

# Create model
model = LinearRegression()
```

```
# Train model
model.fit(X, y)

# Predict price
new_car = [[2022, 25000]]

prediction = model.predict(new_car)

print("CAR PRICE PREDICTION")
print("--------------------")
print("Car Year:", new_car[0][0])
print("Kilometers:", new_car[0][1])
print("Predicted Price: Rs.", round(prediction[0], 2))
```

OUTPUT:



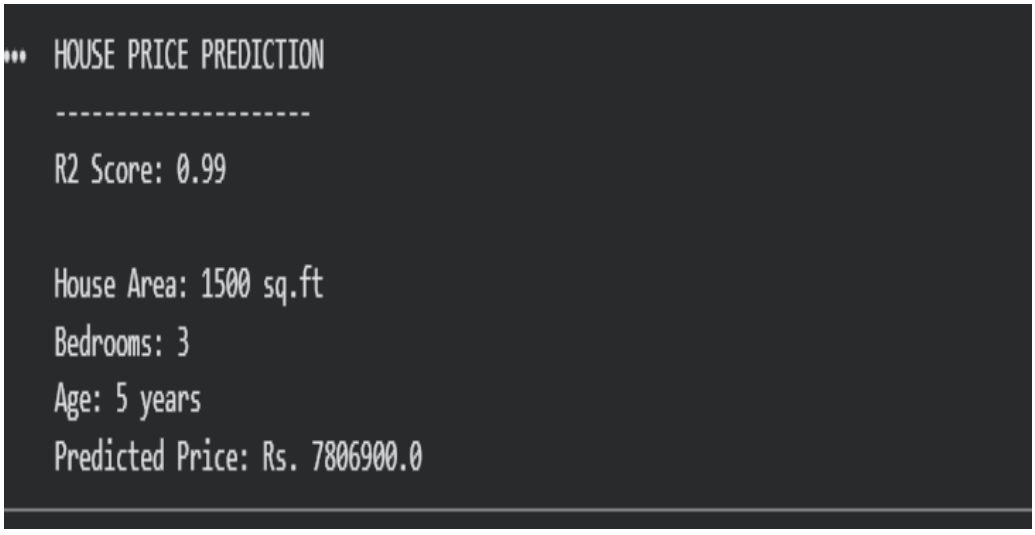
EXP 14:

CODE:

```
# EXPERIMENT 14
# HOUSE PRICE PREDICTION
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score
np.random.seed(42)
# Generate house data
area = np.random.randint(500, 3000, 200)
bedrooms = np.random.randint(1, 6, 200)
age = np.random.randint(1, 30, 200)

# Generate price
price = (
    area * 5000
    + bedrooms * 200000
    - age * 30000
)

X = np.column_stack((area, bedrooms, age))
y = price

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Model
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
```

```
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("HOUSE PRICE PREDICTION")
print("----------------------")
print("R2 Score:", round(r2_score(y_test, y_pred), 2))

# New house
new_house = [[1500, 3, 5]]

prediction = model.predict(new_house)

print("\nHouse Area:", new_house[0][0], "sq.ft")
print("Bedrooms:", new_house[0][1])
print("Age:", new_house[0][2], "years")
print("Predicted Price: Rs.", round(prediction[0], 2))
```

OUTPUT:



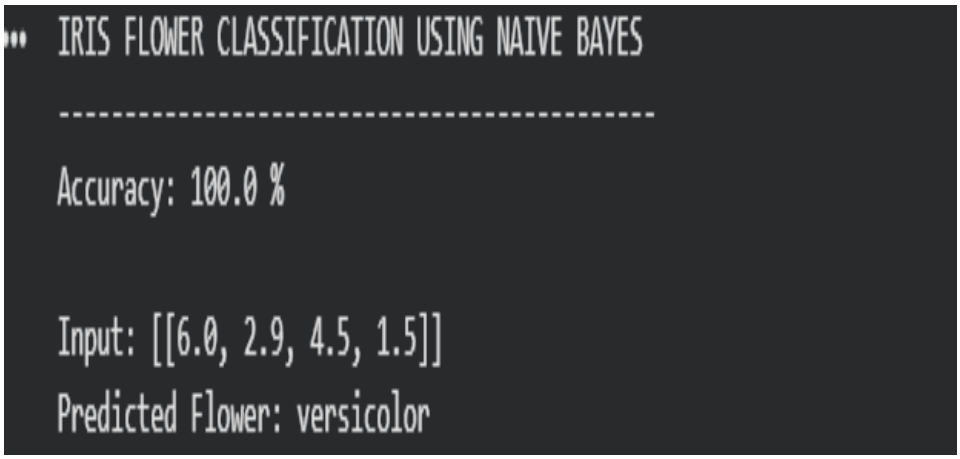
EXP 15:

CODE:

```
# EXPERIMENT 15
# IRIS FLOWER CLASSIFICATION USING NAIVE BAYES

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create Naive Bayes model
model = GaussianNB()

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print("IRIS FLOWER CLASSIFICATION USING NAIVE BAYES")
print("-------------------------------------------")
print("Accuracy:", round(accuracy * 100, 2), "%")

# Test sample
sample = [[6.0, 2.9, 4.5, 1.5]]

prediction = model.predict(sample)

print("\nInput:", sample)
print("Predicted Flower:", iris.target_names[prediction[0]])
```

OUTPUT:



EXP 16:

CODE:

```
# EXPERIMENT 16
# COMPARISON OF CLASSIFICATION ALGORITHMS

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

# Models
models = [
    ("Logistic Regression", LogisticRegression(max_iter=200)),
    ("KNN", KNeighborsClassifier(n_neighbors=5)),
    ("Decision Tree", DecisionTreeClassifier(random_state=42)),
    ("Random Forest", RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )),
    ("SVM", SVC()),
    ("Naive Bayes", GaussianNB())
]

print("CLASSIFICATION ALGORITHM COMPARISON")
print("-----------------------------------")

# Train and test
for name, model in models:

    if name in ["Logistic Regression", "KNN", "SVM"]:
        model.fit(X_train_scaled, y_train)
        prediction = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        prediction = model.predict(X_test)

    accuracy = accuracy_score(y_test, prediction)

    print(
        name,
        "Accuracy =",
        round(accuracy * 100, 2),
        "%"
    )
```

OUTPUT:

```
CLASSIFICATION ALGORITHM COMPARISON
-----------------------------------
Logistic Regression Accuracy = 100.0 %
KNN Accuracy = 100.0 %
Decision Tree Accuracy = 100.0 %
Random Forest Accuracy = 100.0 %
SVM Accuracy = 100.0 %
Naive Bayes Accuracy = 100.0 %
```

EXP 17:

CODE:

```
# EXPERIMENT 17
# MOBILE PRICE PREDICTION

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

np.random.seed(42)

# Generate mobile specifications
ram = np.random.randint(2, 13, 500)
battery = np.random.randint(2000, 6000, 500)
storage = np.random.choice(
    [32, 64, 128, 256, 512],
    500
)
camera = np.random.randint(8, 109, 500)

# Calculate mobile score
score = (
    ram * 3000
    + battery * 5
```

```
    + storage * 100
    + camera * 100
)

# Create price categories
price_range = np.where(
    score < 40000, 0,
    np.where(
        score < 60000, 1,
        np.where(score < 80000, 2, 3)
    )
)

# Features
X = np.column_stack((
    ram,
    battery,
    storage,
    camera
))

y = price_range

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

```
# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("MOBILE PRICE PREDICTION")
print("-----------------------")
print("Accuracy:", round(accuracy * 100, 2), "%")

# New mobile
new_mobile = [[8, 5000, 128, 50]]

prediction = model.predict(new_mobile)[0]

price_categories = [
    "Low",
    "Medium-Low",
    "Medium-High",
    "High"
]

print("\nMobile Specifications:")
print("RAM:", new_mobile[0][0], "GB")
print("Battery:", new_mobile[0][1], "mAh")
print("Storage:", new_mobile[0][2], "GB")
```

OUTPUT:

```
MOBILE PRICE PREDICTION
-----------------------
Accuracy: 96.0 %

Mobile Specifications:
RAM: 8 GB
Battery: 5000 mAh
Storage: 128 GB
Camera: 50 MP
Predicted Price Category: Medium-High
```

EXP 18:

CODE:

```
# EXPERIMENT 18
# PERCEPTRON BASED IRIS CLASSIFICATION

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Standardization
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Perceptron model
model = Perceptron(
    max_iter=1000,
    eta0=0.1,
    random_state=42
```

```
)

# Train
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("PERCEPTRON BASED IRIS CLASSIFICATION")
print("-------------------------------------")
print("Accuracy:", round(accuracy * 100, 2), "%")

# Test sample
sample = [[5.1, 3.5, 1.4, 0.2]]

sample_scaled = scaler.transform(sample)

prediction = model.predict(sample_scaled)

print("\nInput:", sample)
print("Predicted Flower:", iris.target_names[prediction[0]])
```

OUTPUT:

EXP 19:

CODE:

```
# EXPERIMENT 19
# NAIVE BAYES CLASSIFICATION FOR BANK LOAN PREDICTION

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

np.random.seed(42)

# Generate bank customer data
income = np.random.randint(20000, 100000, 500)
loan_amount = np.random.randint(10000, 400000, 500)
credit_score = np.random.randint(300, 850, 500)
employment_years = np.random.randint(1, 20, 500)

# Loan approval rule
approval_score = (
    income
    + credit_score * 100
    + employment_years * 2000
    - loan_amount * 0.2
)

loan_status = np.where(
    approval_score > 60000,
    1,
    0
)

# Features
X = np.column_stack((
    income,
```

```
    loan_amount,
    credit_score,
    employment_years
))

y = loan_status

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Naive Bayes model
model = GaussianNB()

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("BANK LOAN PREDICTION USING NAIVE BAYES")
print("----------------------------------------")
print("Accuracy:", round(accuracy * 100, 2), "%")

# New customer
new_customer = [[60000, 150000, 750, 8]]

prediction = model.predict(new_customer)[0]

print("\nCustomer Details:")
print("Income: Rs.", new_customer[0][0])
```

```
print("Loan Amount: Rs.", new_customer[0][1])
print("Credit Score:", new_customer[0][2])
print("Employment:", new_customer[0][3], "years")

if prediction == 1:
    print("Loan Status: APPROVED")
else:
    print("Loan Status: NOT APPROVED")
```

OUTPUT:

```
BANK LOAN PREDICTION USING NAIVE BAYES
-------------------------------------
Accuracy: 94.0 %

Customer Details:
Income: Rs. 60000
Loan Amount: Rs. 150000
Credit Score: 750
Employment: 8 years
Loan Status: APPROVED
```

EXP 20:

CODE:

```
# EXPERIMENT 20
# FUTURE SALES PREDICTION

import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Create sales data
days = np.arange(1, 31)

sales = np.array([
    100, 105, 110, 115, 120,
    125, 130, 135, 140, 145,
```

```
    150, 155, 160, 165, 170,
    175, 180, 185, 190, 195,
    200, 205, 210, 215, 220,
    225, 230, 235, 240, 245
])

# Convert into 2D array
X = days.reshape(-1, 1)
y = sales

# Create Linear Regression model
model = LinearRegression()

# Train
model.fit(X, y)

# Predict training data
prediction = model.predict(X)

# Accuracy
accuracy = r2_score(y, prediction)

print("FUTURE SALES PREDICTION")
print("-----------------------")
print("R2 Score:", round(accuracy, 2))

# Predict next 7 days
future_days = np.arange(31, 38).reshape(-1, 1)

future_sales = model.predict(future_days)

print("\nPredicted Future Sales:")

for day, sale in zip(
    future_days.flatten(),
    future_sales
```

```
):
    print(
        "Day", day,
        "-> Predicted Sales:",
        round(sale, 2)
    )
```

OUTPUT:

```
FUTURE SALES PREDICTION
----------------------
R2 Score: 1.0

Predicted Future Sales:
Day 31 -> Predicted Sales: 250.0
Day 32 -> Predicted Sales: 255.0
Day 33 -> Predicted Sales: 260.0
Day 34 -> Predicted Sales: 265.0
Day 35 -> Predicted Sales: 270.0
Day 36 -> Predicted Sales: 275.0
Day 37 -> Predicted Sales: 280.0
```